

Name: Sanidhya Negi

Sap Id: 590022442

Batch :19

WEEK – 2

Design and Analysis of Algorithms

LAB ASSIGNMENT

Question 1

Take a random input of 1000 numbers and compare the performance of Linear Search and Binary Search.

The program generates random integers, sorts the array for binary search, chooses an existing value as the search key, and measures the time required by both search techniques.

Question 1 — C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000

int compare(const void *p, const void *q)
{
    return (*(int *)p - *(int *)q);
}

int linearSearch(int arr[], int size, int key)
{
    for (int i = 0; i < size; i++)
        if (arr[i] == key)
            return i;

    return -1;
}

int binarySearch(int arr[], int size, int key)
{
    int low = 0, high = size - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)
            return mid;

        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int arr[SIZE], key;
    clock_t start;

    srand((unsigned)time(NULL));

    for (int i = 0; i < SIZE; i++)
        arr[i] = rand() % 10000;

    qsort(arr, SIZE, sizeof(int), compare);
    key = arr[rand() % SIZE];

    start = clock();
    linearSearch(arr, SIZE, key);
    printf("Linear Search: %.6f sec\n",
        (double)(clock() - start) / CLOCKS_PER_SEC);

    start = clock();
    binarySearch(arr, SIZE, key);
    printf("Binary Search: %.6f sec\n",
        (double)(clock() - start) / CLOCKS_PER_SEC);

    return 0;
}
```

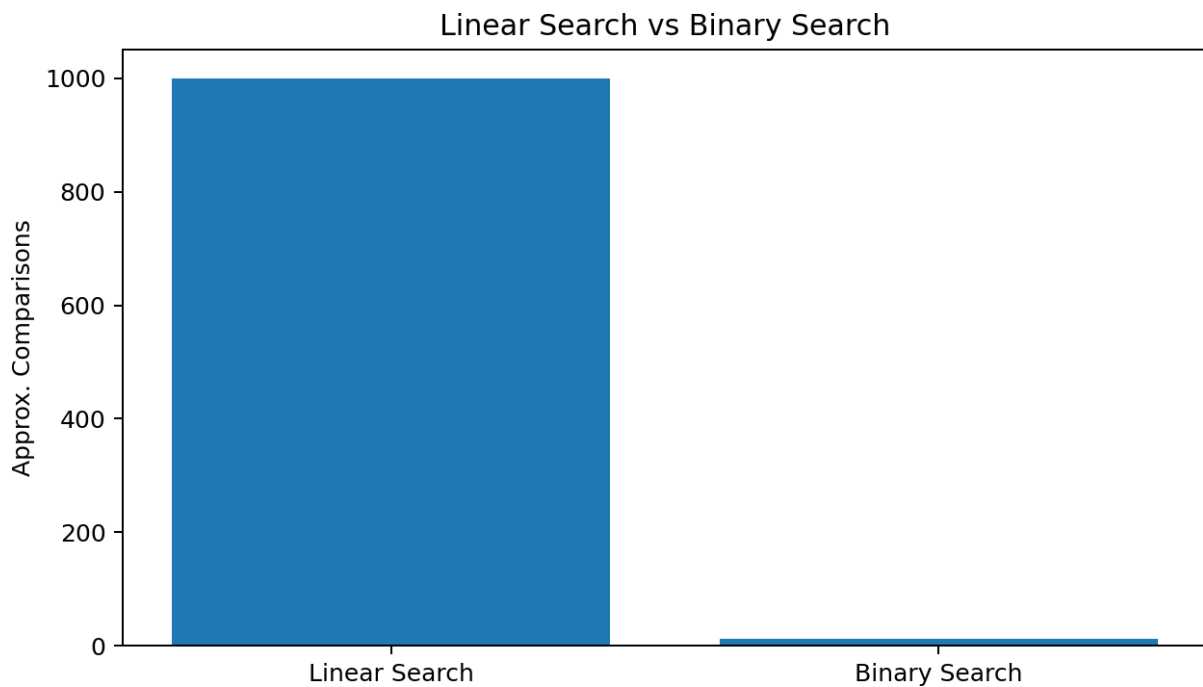
Note: With only 1000 elements, both searches may complete extremely quickly, so one timing measurement can be too small to show a noticeable difference. A better timing experiment is to repeat each search many times, for example 100,000 times.

Question 1 — Explanation and Result

compare() function: This function is passed to **qsort()** to arrange the array in ascending order. The **void *** parameters can represent any pointer type. They are converted to integer pointers, and the actual integer values are then compared.

How the searches differ: Linear Search checks values sequentially until the target is found. Binary Search requires sorted data and repeatedly divides the current search interval into two halves.

Time complexity: Linear Search requires **$O(n)$** time, whereas Binary Search requires **$O(\log n)$** time. For 1000 elements, Binary Search therefore needs substantially fewer comparisons in general.



The graph visually compares the number of comparisons made by the two search methods.

Question 2

Write a C program to generate 1000 random numbers and compare the execution time of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. Display the time taken by each sorting algorithm and plot a comparison graph using Python.

The same original random array is copied before each test so that every sorting method receives identical input data.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000

void bubbleSort(int a[])
{
    for (int i = 0; i < SIZE - 1; i++)
        for (int j = 0; j < SIZE - i - 1; j++)
            if (a[j] > a[j + 1])
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
}

void selectionSort(int a[])
{
    for (int i = 0; i < SIZE - 1; i++)
    {
        int pos = i;

        for (int j = i + 1; j < SIZE; j++)
            if (a[j] < a[pos])
                pos = j;

        int temp = a[i];
        a[i] = a[pos];
        a[pos] = temp;
    }
}

void insertionSort(int a[])
{
    for (int i = 1; i < SIZE; i++)
    {
        int value = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > value)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = value;
    }
}
```

Question 2 — C Program (continued)

```
void mergeSort(int a[], int left, int right)
{
    if (left >= right)
        return;

    int mid = (left + right) / 2;
    int temp[SIZE];

    mergeSort(a, left, mid);
    mergeSort(a, mid + 1, right);

    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right)
        temp[k++] = (a[i] < a[j]) ? a[i++] : a[j++];

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (i = left; i <= right; i++)
        a[i] = temp[i];
}

void quickSort(int a[], int left, int right)
{
    if (left >= right)
        return;

    int i = left, j = right;
    int pivot = a[(left + right) / 2];

    while (i <= j)
    {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;

        if (i <= j)
        {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
            j--;
        }
    }

    quickSort(a, left, j);
    quickSort(a, i, right);
}

int main()
{
    int original[SIZE], copy[SIZE];
    clock_t start;

    srand((unsigned)time(NULL));

    for (int i = 0; i < SIZE; i++)
        original[i] = rand() % 10000;

    for (int method = 0; method < 5; method++)
    {
        for (int i = 0; i < SIZE; i++)
            copy[i] = original[i];

        start = clock();

        if (method == 0) bubbleSort(copy);
```

```

        else if (method == 1) selectionSort(copy);
        else if (method == 2) insertionSort(copy);
        else if (method == 3) mergeSort(copy, 0, SIZE - 1);
        else quickSort(copy, 0, SIZE - 1);

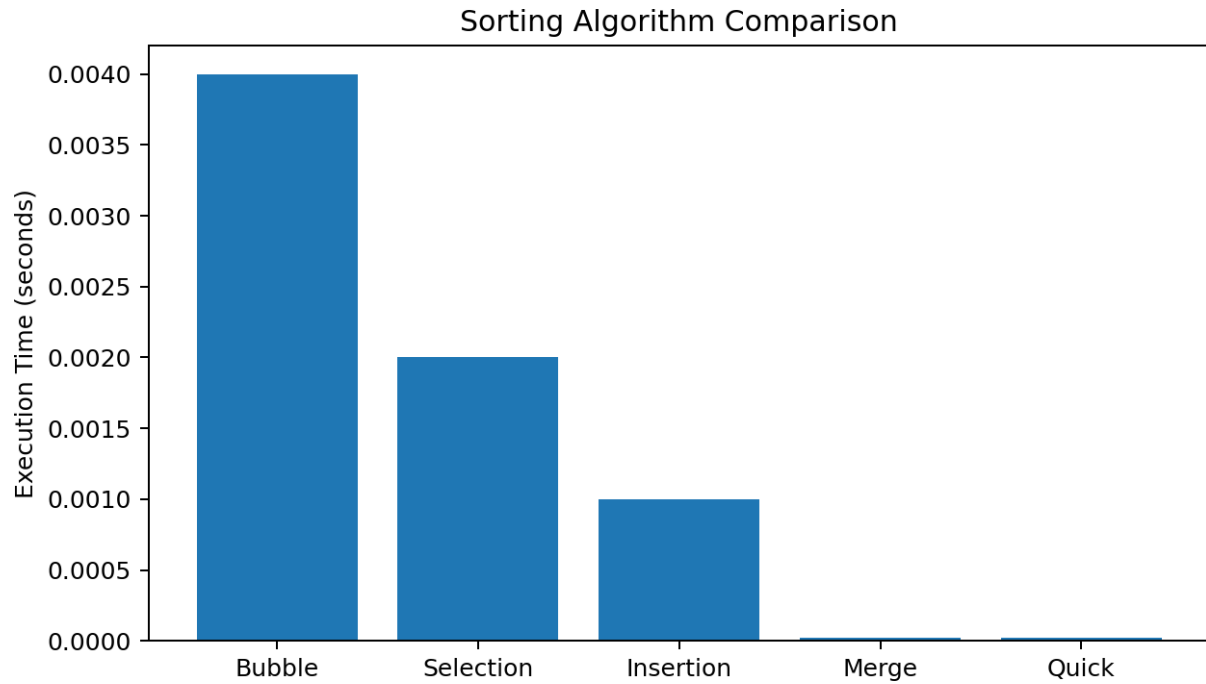
        printf("%.6f seconds\n",
               (double)(clock() - start) / CLOCKS_PER_SEC);
    }

    return 0;
}

```

The execution time is obtained with **clock()**. Actual timing values can differ between computers and between runs because they depend on the execution environment.

Question 2 — Graph and Conclusion



Conclusion: For a large collection of random elements, Merge Sort and Quick Sort are generally faster than Bubble Sort, Selection Sort, and Insertion Sort. Bubble Sort, Selection Sort, and Insertion Sort have $O(n^2)$ average/worst-case complexity. Merge Sort has $O(n \log n)$ complexity, while Quick Sort has an average complexity of $O(n \log n)$. Thus, Merge Sort and Quick Sort are more efficient choices for large datasets.

Time Complexity Comparison Table

Sorting Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Overall observation: As the input size grows, the choice of algorithm becomes increasingly important. Algorithms with $O(n \log n)$ growth scale better than algorithms whose running time grows quadratically.

— END OF WEEK 2 ASSIGNMENT —